

ESTADO: FRONTEND 100%
VALIDADO

ATLAS GYM

Auditoría de Sistemas & Recomendaciones Técnicas para Backend y Base de Datos

Fecha: 24 de Septiembre de 2026
Destinatario: Bruno / Equipo ATLAS GYM
Rama Git: ramaBruno (Commits locales al día)

01. Resumen Ejecutivo

El frontend de **ATLAS Gym** ha concluido satisfactoriamente su etapa de pulido integral y verificación funcional, contando con **108 tests unitarios pasando al 100%** y compilación en Turbopack libre de errores. Con la resolución de la sincronización en vivo entre Finanzas y Asistencias y el bloqueo de registros duplicados en el Tótem, el sistema cuenta con todas sus interfaces operativas listas para conectarse a un backend formal.

El propósito de este documento es detallar la **arquitectura de base de datos relacional**, las **reglas de negocio de servidor** y las **prácticas de seguridad de grado industrial** necesarias para garantizar que el traspaso a PostgreSQL/NodeJS sea fluido, escalable y robusto.

Principio Rector de la Transición: El frontend actual ya implementa las interfaces y contratos de datos definitivos (Alumno , Pago , Plan , Rutina , RegistroAsistencia , Aviso). La migración consistirá en reemplazar las operaciones locales de lib/store.tsx por peticiones HTTP/REST a controladores autenticados, sin necesidad de reescribir las pantallas de usuario.

02. Estado Actual y Activos del Frontend

Módulo Frontend	Comportamiento Implementado	Impacto / Preparación Backend
Asistencias & Tótem	Check-in táctil por DNI, caducidad de aforo a 100 min, prevención de doble ingreso diario y sincronización inter-pestañas.	Requiere tabla asistencias indexada por (alumno_id, fecha) con constraint UNIQUE condicional.
Finanzas & Pagos	Cálculo de vencimientos al día 10, morosidad mensual automática, buscador por nombre/DNI y generación de link WhatsApp.	Requiere tabla pagos con snapshot inmutable de precios y tabla planes .
Rutinas & Alumnos	Asignación múltiple con checkboxes, timer de descanso con sonido Web Audio API y biblioteca de 1.500 GIFs biomecánicos.	Requiere tablas rutinas , dias_rutina , ejercicios y tabla puente alumno_rutinas .
Avisos & Notificaciones	Mensajería global ('TODOS') y personalizada, conteo de no leídos en sidebar y auto-marcado al visualizar.	Requiere tabla avisos y tabla de lecturas aviso_lecturas (aviso_id, usuario_id) .

03. Recomendaciones de Arquitectura & Base de Datos

3.1. Inmutabilidad de Pagos e Historial de Precios (Auditoría Contable)

En un gimnasio, los precios de las membresías cambian a lo largo del año por inflación o políticas comerciales. Si se actualiza el precio del plan Full Access de \$22.000 a \$26.000 en el backend, los pagos pasados o las cuotas

emitidas con anterioridad no deben recalcularse retroactivamente.

Recomendación para la BD: La tabla `pagos` debe almacenar explícitamente: `monto_cobrado (DECIMAL)`, `precio_plan_momento (DECIMAL)`, `periodo_mes ("YYYY-MM")`, `metodo_pago` y `fecha_pago (DATE)`. No debe confiarse únicamente en un `JOIN` dinámico a la tabla `planes` para saber cuánto costaba el pase en meses anteriores.

3.2. Reglas de Asistencia: Control Anti-Fraude vs. Doble Turno

El control implementado en el frontend bloquea más de un ingreso el mismo día calendario. Al diseñar el backend, la lógica recomendada es:

- **Regla por defecto (98% de los casos):** Impedir nuevo check-in si existe un registro para el socio con `fecha = CURRENT_DATE` y la sesión previa tiene menos de 100 minutos de antigüedad (sesión activa en sala).
- **Excepción para doble turno:** Si el gimnasio incorpora socios con pases especiales de doble turno (ej: mañana musculación y tarde spinning), el backend puede permitir un segundo check-in *únicamente* si la sesión anterior ya ha superado los 100 minutos (`timestamp < NOW() - INTERVAL '100 minutes'`) y el plan lo habilita.
- **Índice en BD:** Crear índice compuesto `CREATE INDEX idx_asistencias_alumno_fecha ON asistencias (alumno_id, fecha);` para que la verificación en el tótem responda en menos de 5 milisegundos.

ATLAS GYM — Guía de Arquitectura de Backend

Página 2 / 2

3.3. Autenticación y Control de Accesos Basado en Roles (RBAC + JWT)

Actualmente el frontend bifurca rutas en el cliente mediante `AppShell` y `puedeAccederRuta`. En el backend, la seguridad debe ser infalible a nivel API:

- **Tokens JWT sin estado:** El token de sesión debe contener `{ sub: usuario_id, rol: "ADMIN" | "ALUMNO", alumno_id: string | null }` firmado con secreto seguro (HS256 o RS256).
- **Protección de endpoints por Guard/Middleware:**
 - `POST /api/pagos`, `PUT /api/planes`, `DELETE /api/alumnos/:id` → Exclusivos para rol `ADMIN`.
 - `GET /api/mis-cuotas`, `GET /api/mi-rutina` → Autolimitados al `req.user.alumno_id` para evitar fuga de datos entre socios.
 - `POST /api/asistencias/totem` → Endpoint protegido por API Key dedicada o token de dispositivo exclusivo de la tablet de recepción.

3.4. Sincronización en Tiempo Real (De BroadcastChannel a WebSockets / SSE)

En la versión local actual utilizamos `BroadcastChannel` y `StorageEvent`, lo cual sincroniza perfectamente diferentes pestañas del mismo navegador. Al pasar a producción en red:

Opción A: Server-Sent Events (SSE)

Unidireccional servidor → navegador. Ideal para el panel de Asistencias y Aforo. Extremadamente liviano, reconexión automática nativa en HTTP/2 sin overhead.

RECOMENDADO PARA ASISTENCIAS

Opción B: WebSockets (Socket.io)

Bidireccional en tiempo real. Conveniente si a futuro se añade chat interactivo profesor-alumno o notificaciones push instantáneas en vivo.

ALTERNATIVA ESCALABLE

3.5. Transacciones ACID e Idempotencia en Cobros y Check-ins

En operaciones concurrentes (por ejemplo, dos recepcionistas cobrando al mismo tiempo o un socio ingresando mientras se procesa su pago):

- **Transacciones atómicas:** El cobro de una cuota y la actualización del estado del socio deben ejecutarse dentro de un bloque `BEGIN ... COMMIT` para evitar que un fallo de red deje un pago registrado sin actualizar la cuenta del alumno.
- **Claves de Idempotencia:** En el endpoint de cobro, aceptar un encabezado `X-Idempotency-Key` generado por el cliente. Si la recepcionista hace doble clic accidental en "Guardar pago", la API rechazará el duplicado.

04. Hoja de Ruta Sugerida de Implementación (Roadmap)

Fase 1: Modelado de BD y Migraciones (Día 1 a 3)

Creación del esquema relacional en PostgreSQL / Supabase: tablas `usuarios`, `alumnos`, `planes`, `pagos`, `rutinas`, `ejercicios`, `asistencias` y `avisos`.

Fase 2: API REST Core & Autenticación JWT (Día 4 a 7)

Desarrollo de endpoints de autenticación, CRUD de alumnos, módulo de finanzas con cálculo de vencimientos al día 10 y endpoint del totem.

Fase 3: Conexión Frontend & Adaptador de Store (Día 8 a 10)

Reemplazo de los métodos locales de `useGymStore` en `lib/store.tsx` por llamadas a la API usando React Query o SWR, manteniendo intactos los componentes de vista.

Fase 4: Eventos en Vivo & Despliegue (Día 11 a 12)

Configuración de SSE/WebSockets para el aforo en sala de recepción y puesta en producción en servidor VPS o Cloud (Docker, Railway o AWS).

05. Checklist de Criterios de Aceptación

- ☒ **Frontend 100% verificado:** 108 tests unitarios pasando, build Turbopack limpio.
- ☒ **Sincronización Finanzas ↔ Asistencias:** Impacto inmediato al cobrar en recepción.
- ☒ **Prevención de duplicados en Tótem:** Feedback visual y sonoro con preservación del aforo.
- ☒ **Diseño y UX:** Sentence case global, tipografía estandarizada en `md:text-4xl`, tooltips en Oms y confirmación de cierre de sesión.
- ☐ **Siguiente paso:** Inicio del desarrollo del Backend y esquema de base de datos relacional.

Documento generado automáticamente para ATLAS Gym — Sistema de Gestión Integral de Gimnasio. Todos los derechos reservados © 2026.